

Compilerbau

Aufgabenstellungen der Übungsblätter

Blatt	Thema	Punkte
0	Aufsetzen der Rust-Entwicklungsumgebung	-
1	Syntree: Parser, Pretty-Printer und Interpreter für einen Taschenrechner	100 (40 / 20 / 40)
2	Lexer für C1 mit logos	-
3	Parser für C1 mit lalrpop	-
4	Semantische Analyse	100
5	Interpreter	-

Zusammengestellt aus den Aufgabentexten der Moodle-Seiten. Verweise auf Moodle-Seiten sind als Text wiedergegeben.

Inhalt

Blatt 0 – Aufsetzen der Rust-Entwicklungsumgebung	3
Blatt 1 – Syntree: Parser, Pretty-Printer und Interpreter	6
Blatt 2 – Lexer für C1	9
Blatt 3 – Parser für C1	11
Blatt 4 – Analyse (100 Punkte)	14
Blatt 5 – Interpreter	17

Blatt 0 - Aufsetzen der Rust-Entwicklungsumgebung

Die folgenden Schritte beziehen sich auf Ubuntu Linux. Unter Windows und macOS ist die Installation analog möglich; plattformspezifische Unterschiede sind jeweils gekennzeichnet. Bei Problemen melden Sie sich bitte in der Übung.

Schritt 1: Rust-Toolchain installieren

Die Rust-Toolchain umfasst drei Werkzeuge:

rustc	Der Rust-Compiler.
cargo	Build-System und Paketmanager — vergleichbar mit make + Abhängigkeitsverwaltung.
rustup	Verwaltet Toolchain-Versionen und ermöglicht parallele Installationen.

Linux

Öffnen Sie ein Terminal und führen Sie folgenden Befehl aus:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Folgen Sie den Anweisungen im Installer (die Standardoptionen sind in der Regel passend). Danach müssen Sie sich aus- und wieder einloggen oder alternativ in der aktuellen Shell folgenden Befehl ausführen:

```
source "$HOME/.cargo/env"
```

Windows

Rust benötigt unter Windows einen C/C++-Linker. Installieren Sie zunächst die Microsoft C++ Build Tools. Im Installer genügt die Auswahl von **Desktopentwicklung mit C++** (inkl. Windows SDK). Laden Sie anschließend rustup-init.exe von rustup.rs herunter und führen Sie das Programm aus.

macOS

Installieren Sie die Xcode Command Line Tools über das Terminal:

```
xcode-select --install
```

Danach verläuft die Rust-Installation wie unter Linux (mit dem curl-Befehl oben).

Überprüfung (alle Plattformen)

Prüfen Sie die Installation, indem Sie in einem neuen Terminal eingeben:

```
rustc --version  
cargo --version
```

Beide Befehle sollten eine Versionsnummer ausgeben. Falls stattdessen „command not found“ erscheint, wurde die Pfad-Konfiguration noch nicht übernommen. Loggen Sie sich aus und wieder ein oder starten Sie das Terminal neu.

Schritt 2: Editor einrichten

Für die Arbeit mit Rust empfehlen wir einen der beiden folgenden Editoren. Die Wahl ist Ihnen überlassen — beide bieten Code-Vervollständigung, Inline-Fehlermeldungen, Refactoring und Debugger-Integration für Rust.

Option A: Visual Studio Code + rust-analyzer

Empfehlung: VS Code mit rust-analyzer ist der in der Rust-Community am weitesten verbreitete Editor und bietet hervorragende Unterstützung bei minimalem Ressourcenverbrauch.

1. Laden Sie Visual Studio Code herunter und installieren Sie den Editor. Unter Ubuntu steht auch ein snap-Paket zur Verfügung (`sudo snap install code --classic`).
2. Öffnen Sie VS Code und installieren Sie die Erweiterung **rust-analyzer** (`rust-lang.rust-analyzer`) über die Seitenleiste Extensions (`Ctrl+Shift+X`). Diese Erweiterung stellt bereit:

Code-Vervollständigung	Kontextabhängige Vorschläge für Typen, Methoden und Traits.
Inline-Diagnostik	Compilerfehler und -warnungen direkt im Editor.
Typ-Inlays	Automatisch eingeblendete Typ-Annotationen bei <code>let</code> -Bindungen.
Go-to-Definition	Navigation zu Funktions- und Typdefinitionen mit <code>F12</code> .

3. Optional, aber empfohlen: Installieren Sie zusätzlich die Erweiterungen **CodeLLDB** (`vadimcn.vscode-lldb`) für Debugger-Unterstützung und **Even Better TOML** (`tamasfe.even-better-toml`) für Syntax-Highlighting in `Cargo.toml`.

Option B: JetBrains RustRover

RustRover ist eine eigenständige IDE von JetBrains mit integrierter Rust-Unterstützung. Die nicht-kommerzielle Edition ist für Studierende kostenfrei.

1. Laden Sie RustRover von der Download-Seite herunter. Unter Linux erhalten Sie ein `.tar.gz`-Archiv, unter Windows einen `.exe`-Installer und unter macOS ein `.dmg`-Image.
2. Starten Sie RustRover. Die Rust-Toolchain wird automatisch erkannt, sofern die Installation aus Schritt 1 korrekt abgeschlossen wurde.

Schritt 3: Erstes Projekt erstellen

Öffnen Sie ein Terminal, navigieren Sie in Ihr Projektverzeichnis und erstellen Sie ein neues Cargo-Projekt:

```
cargo new hello_world
cd hello_world
```

Cargo legt folgende Verzeichnisstruktur an:

```
hello_world/
├── Cargo.toml      # Projektmanifest (Name, Version, Abhängigkeiten)
├── src/
│   └── main.rs     # Einstiegspunkt mit einer Hello-World-Funktion
```

Öffnen Sie den Ordner `hello_world` nun in Ihrem Editor: in VS Code mit `code .` im Terminal oder über `File → Open Folder`, in RustRover über `Open` auf dem Startbildschirm.

Öffnen Sie `src/main.rs`. Sie sollten den Quellcode dank Syntax-Highlighting farblich hervorgehoben sehen:

```
fn main() {
    println!("Hello, world!");
}
```

Übersetzen und starten Sie das Programm über das Terminal:

```
cargo run
```

Die Ausgabe sollte in etwa so aussehen:

```
Compiling hello_world v0.1.0 (/home/user/hello_world)
Finished `dev` profile [unoptimized + debuginfo] target(s)
Running `target/debug/hello_world`
Hello, world!
```

Alternativ können Sie das Programm in beiden Editoren auch direkt ausführen: In VS Code über das Terminal-Panel (Ctrl+`) oder mit der Erweiterung CodeLLDB über F5. In RustRover über den grünen Play-Button neben der `main()`-Funktion.

Schritt 4: Debugger (optional)

Beide Editoren unterstützen das Debugging von Rust-Programmen über LLDB oder GDB. Um einen Haltepunkt zu setzen, klicken Sie links neben eine Zeilennummer und starten Sie das Programm im Debug-Modus (F5 in VS Code bzw. das Käfer-Symbol in RustRover).

Gut zu wissen: Dank des strengen Typsystems und des Borrow-Checkers benötigen Sie den Debugger in Rust erfahrungsgemäß deutlich seltener als in anderen Sprachen — wenn das Programm kompiliert, funktioniert es häufig auch korrekt.

Blatt 1 - Syntree: Parser, Pretty-Printer und Interpreter

Lernziele

- Konstruktion einer Datenstruktur aus einem arithmetischen Ausdruck
- methodisches Ablaufen einer rekursiven Datenstruktur
- Umwandlung einer Datenstruktur in eine Zeichenkette
- Interpretation eines abstrakten Parsebaums

Generelles

In diesem Übungsblatt entwickeln Sie die stark-vereinfachte Variante eines Interpreters für eine kleine Taschenrechnersprache. Dabei durchlaufen Sie konzeptionell alle Phasen der Compilerentwicklung, jedoch ohne Nutzung der Techniken und Werkzeuge, die Sie im Laufe des Semesters noch kennenlernen. Das gibt Ihnen die Gelegenheit ein mentales Modell der bevorstehenden Herausforderungen zu entwickeln und erlaubt Ihnen gleichzeitig einen sanften Einstieg in die Implementationssprache Rust.

Sie können sich aussuchen, ob Sie dieses Übungsblatt in Rust oder in C lösen möchten. Die Variante für C finden Sie hier (Moodle).

Allgemeine Hinweise zur Bearbeitung und zum Abgabemodus finden Sie auf dieser Seite (Moodle).

Kurzbeschreibung

Sie erhalten die Datenstrukturen eines Parsebaums, der Ausdrücke in umgekehrter polnischer Notation mit Operanden aus der Menge der Ziffern (für Zahlenliterale) sowie aus der Menge der kleingeschriebenen ASCII-Buchstaben (für Variablenbezeichner) und den Operatoren ['+', '-', '*', '/'] repräsentieren kann. Darüber hinaus soll die Belegung von Variablen mithilfe des Zuweisungsoperators '=' möglich sein.

Ihre Aufgabe besteht darin, für diese Datenstruktur einen Parser, einen Pretty-Printer und einen Interpreter zu implementieren.

Vorlage

Um Ihnen den Einstieg in diese Aufgabe zu vereinfachen, geben wir Ihnen ein Programmgerüst vor. Die Ordnerstruktur des entpackten Archives ist wie folgt:

```
.
├── src
│   ├── calculator.rs
│   ├── lib.rs
│   ├── main.rs
│   ├── parse_tree.rs
│   └── printer.rs
├── Cargo.lock
└── Cargo.toml
```

Die Dateien `main.rs` und `lib.rs` sowie die `Cargo.toml` müssen Sie nicht anpassen. Sie enthalten den Programmeinstiegspunkt (`main.rs`), exponieren die öffentliche Schnittstelle (`lib.rs`) und Metainformationen über das Projekt (`Cargo.toml`).

- In `parse_tree.rs` befinden sich die Datenstrukturen des Parsebaums, das Visitor-Trait (ein Beispiel für das Visitor-Pattern) sowie der Einstiegspunkt des Parsers.
- In `printer.rs` befindet sich der Einstiegspunkt für den Pretty-Printer.
- In `calculator.rs` finden Sie den Einstiegspunkt des Interpreters.

Diese drei Dateien enthalten auch Unit-Tests, die Sie nach Ruf von `cargo test` bei der Fehlersuche unterstützen.

Implementation des Parsers (40 Punkte)

Die Funktion zum Parsen finden Sie in der Quelldatei `parse_tree.rs`. Der Parser soll Ausdrücke in umgekehrter polnischer Notation akzeptieren und daraus einen Parsebaum konstruieren, der den eingelesenen Ausdruck repräsentiert. Ggf. auftretende Fehler sollen erkannt und in Form einer der drei möglichen Fehlerarten `Error::Lexical`, `Error::Syntax` oder `Error::Semantic` zurückgegeben werden.

Beispielausdrücke

Wie oben erwähnt, soll der Parser Ausdrücke in umgekehrter polnischer Notation akzeptieren.

Ausdruck	Infix Notation
1 2 +	1 + 2
1 2 3 + *	1 * (2 + 3)
a 1 =	a = 1
a 1 = a a +	a = 1; a + a

Folgende Ausdrücke sollen einen Fehler produzieren:

Ausdruck	gemeldete Fehlerart
1 2 ^	lexikalisch
1 2 3 +	syntaktisch
1 2 + *	syntaktisch
1 1 =	semantisch

Welche Fehlerart diagnostiziert wird, hängt konzeptionell von der Übersetzungsphase ab, in der der entsprechende Fehler detektiert wird. Da in diesem einfachen Beispiel Scanner, Parser und Typprüfung zusammenfallen, wählen wir hier die Konvention, dass lexikalische Fehler durch unbekannte Zeichen im Eingabestrom hervorgerufen werden, syntaktische Fehler durch die falsche Anzahl an Operanden bzw. Operatoren entstehen und semantische Fehler die Folge von Zuweisungen an nicht-zuweisungsfähige Strukturen sind.

Diese Konvention erscheint willkürlich, wird aber in den folgenden Aufgabenblättern verständlich.

Implementation des Pretty-Printers (20 Punkte)

Implementieren Sie einen Pretty-Printer für den Parsebaum, indem Sie die markierte Methode in der Datei `printer.rs` umsetzen. Nutzen Sie dabei das Visitor-Trait und implementieren Sie es entsprechend. Sie müssen keine Methoden überschreiben, die bereits das von Ihnen gewünschte Verhalten aufweisen, insbesondere wird in der Default-Implementation dieses Traits bereits der gesamte Parsebaum abgelaufen.

Der Printer soll die Ausdrücke des Parsebaums hierarchisch geklammert ausgeben. Zuweisungen sollen jeweils durch ein Semikolon getrennt werden.

Beispielausgaben

Eingabe	erwartete Ausgabe
1 1 +	(1+1)
1 2 3 + *	(1*(2+3))
a 1 =	a=1
a 1 = a a +	a=1; (a+a)

Implementation des Interpreters (40 Punkte)

Implementieren Sie einen Interpreter für den Parsebaum, indem Sie die markierte Methode in der Datei `calculator.rs` umsetzen. Nutzen Sie auch dabei das Visitor-Trait.

Der Interpreter soll den Parsebaum gemäß der umgekehrten polnischen Notation ausrechnen. Dabei sollen auch Variablen verwendet und zugewiesen werden können. Als Bezeichner sind nur die Buchstaben 'a' bis 'z' möglich, es gibt also nur 26 mögliche Variablennamen. Variablen werden durch einen Set-Knoten gesetzt und durch einen Var-Knoten gelesen.

Beispielausgaben

Eingabe	erwartetes Ergebnis
1 1 +	2
1 2 3 + *	5
a 1 = a a +	2

Wir wünschen Ihnen viel Spaß und Erfolg bei der Bearbeitung dieses Übungsblattes.

Blatt 2 - Lexer für C1

Lernziele

- Bildung regulärer Ausdrücke anhand informaler Beschreibungen
- Identifikation lexikalischer Fehler
- Umgang mit Metawerkzeugen zur Generierung von Lexern

Generelles

In diesem Übungsblatt entwickeln Sie unter Nutzung des Metawerkzeugs logos einen Lexer für die Token der Sprache C1. Damit schaffen Sie die Grundlage für die weitere Verarbeitung durch den Parser.

Sie können sich aussuchen, ob Sie dieses Übungsblatt in Rust oder in C lösen möchten. Die Variante für C finden Sie hier (Moodle).

Allgemeine Hinweise zur Bearbeitung und zum Abgabemodus finden Sie auf dieser Seite (Moodle).

Aufgabenstellung

Implementieren Sie mithilfe von logos einen Lexer, der in einem Eingabestrom bzw. in einer Eingabedatei die Token der Sprache C1 erkennt.

Einfache Token

Tokenart	Symbole
Schlüsselworte	do else for if print return while
Datentypen	bool float int void
Arithmetische Operatoren	+ - * / =
Logische Operatoren	&&
Vergleichsoperatoren	== != < > <= >=
Separatoren	, ; () { }
Boolesche Literale	true false

Komplexe Token

Tokenart	Beispiele	Beschreibung
Integer-Literal	13, 001, aber nicht 0x12, 0b100	Nichtleere Folge von Ziffern (0-9)
Zeichenkettenliteral	"Hallo", aber nicht "'Laser'"	Zeichen ohne Zeilenenden oder Anführungszeichen, umschlossen von "..."
Fließkommaliteral	1.0, .01, 3.1e-12 ($3.1 \cdot 10^{-12}$), 2E3 ($2 \cdot 10^3$), aber nicht 1., 2.0e-1.4	Ziffernfolge mit . (nicht an letzter Stelle), optional mit Exponent [eE] [+ -]?Ziffern; bei Exponent ist . optional
Bezeichner	Lexer, is_power_of_two, LOG_2, aber nicht 123foo, is-invalid	Folge aus Buchstaben, Ziffern, _; beginnt nicht mit Ziffer

Zusätzlich sollen Whitespace, Blockkommentare à la C und Zeilenkommentare à la C++ überlesen werden:

Art	Beispiele	Beschreibung
Whitespace	—	Sequenzen von Leerzeichen, Tabulatoren, CR und LF
Blockkommentar	/* */, /**/, aber nicht /*/* *//*/, /*	Beginnt mit /*, endet mit */; keine Schachtelung

Art	Beispiele	Beschreibung
Zeilenkommentar	//, /**	Beginnt mit //, endet mit Zeilenende

Alle Tokenarten, die in dieser Aufzählung nicht vorkommen, sollen zu einem lexikalischen Fehler führen, der die problematische Zeichenkette und die Spanne im Text enthält. Dafür können Sie sich auf die Funktionalität von logos verlassen und müssen diese Informationen nicht selbst sammeln.

Vorlage

Um Ihnen den Einstieg in diese Aufgabe zu vereinfachen, geben wir Ihnen ein Programmgerüst vor. Die Ordnerstruktur des entpackten Archives ist wie folgt:

```

.
├── src
│   ├── lexer.rs
│   ├── lib.rs
│   └── main.rs
├── tests
│   ├── inputs
│   │   └── ok
│   │       ├── demorgan.c1
│   │       └── demorgan.token
│   └── lexer_tests.rs
├── Cargo.lock
└── Cargo.toml

```

Die Dateien `main.rs` und `lib.rs` sowie die `Cargo.toml` müssen Sie nicht anpassen. Sie enthalten den Programmeinstiegspunkt (`main.rs`), exponieren die öffentliche Schnittstelle (`lib.rs`) und Metainformationen über das Projekt (`Cargo.toml`).

- In `lexer.rs` finden Sie die Enumeration `Token`, die sämtliche Tokenarten beinhaltet. Diese Enumeration sollten Sie mithilfe von Logos `Derive`-Annotationen anreichern.
- In `lexer_tests.rs` finden Sie einige vordefinierte Tests für Ihren Lexer. Bitte beachten Sie, dass diese Tests nur einen kleinen Teil aller Fälle abdecken und scheuen Sie sich nicht davor, Ihre eigenen Testfälle zu ergänzen.
- Im `tests/inputs/ok`-Ordner finden Sie ein Beispiel für eine `c1`-Quelldatei, zusammen mit der erwarteten Ausgabe zur Selbstüberprüfung.

Hinweise

Schauen Sie sich das Logos-Handbuch und dort insbesondere die Beispiele an, um eine Idee zu bekommen, wie Sie dieses Werkzeug nutzen können. Bitte beachten Sie, dass unsere Tokendefinitionen nicht genau mit denen aus den Beispielausdrücken übereinstimmen und übernehmen die regulären Ausdrücke nicht einfach blind.

Nutzen Sie die `parse()`-Methode, die für Zeichenketten definiert ist, um für die Literale den entsprechenden Wert abzuleiten. Um etwa von der Zeichenkette `"123"` auf den Integer-Wert `123` zu kommen, können Sie `"123".parse::<i64>()` rufen. Den Zieldatentyp wählen Sie mittels `turbofish`-Notation aus.

Wir wünschen Ihnen viel Spaß und Erfolg bei der Bearbeitung dieses Übungsblattes.

Blatt 3 - Parser für C1

Lernziele

- Syntaktische Analyse von Programmcode
- Umgang mit Mehrdeutigkeiten in kontextfreien Grammatiken
- parsergesteuerte Konstruktion eines abstrakten Syntaxbaumes

Generelles

Implementieren Sie mithilfe des Parsergenerators `lalrpop` (crates.io/crates/lalrpop) einen Parser für die Sprache C1. Der Parsergenerator erwartet dabei einen Tokenstrom des Lexers, prüft dessen Syntax und gibt im Falle eines positiven Ergebnisses eine Instanz des abstrakten Syntaxbaumes zurück. Den auf diese Weise erzeugten Syntaxbaum werden wir in der nächsten Aufgabe einer semantischen Prüfung unterziehen.

Sie können sich aussuchen, ob Sie dieses Übungsblatt in Rust oder in C lösen möchten. Die Variante für C finden Sie hier ([Moodle](#)).

Allgemeine Hinweise zur Bearbeitung und zum Abgabemodus finden Sie auf dieser Seite ([Moodle](#)).

Aufgabenstellung

Wir geben Ihnen eine Vorlage mit dem Lexer und den Datenstrukturen des abstrakten Syntaxbaumes vor und haben den Parser bereits für Sie vorbereitet. Ihre Aufgabe besteht darin, einen Parser für die Sprache C1 zu erstellen, der den Syntaxbaum instanziiert. Erweitern Sie dazu die Regeln der Grammatik für die Sprache C1 in der Vorlage `parser.lalrpop`. Die Datenstrukturen, aus denen Sie den Syntaxbaum aufbauen, sind in der Datei `ast.rs` vorgegeben. Diese Datenstrukturen dürfen Sie nicht verändern, da die vorgegebenen Testfälle ansonsten nicht erfolgreich durchlaufen.

Da `lalrpop` nur deterministisch-kontextfreie LALR(1)-Grammatiken akzeptiert, müssen Sie eventuell auftretende Mehrdeutigkeiten in der C1-Grammatik durch Transformationen auflösen. Dazu dürfen Sie die Grammatik natürlich anpassen, allerdings dürfen sich dadurch weder die akzeptierte Sprache noch der daraus erzeugte abstrakte Syntaxbaum verändern.

Die Grammatik von C1 finden Sie nachfolgend:

```

program          ::= item*;
item             ::= declassignment ";" | functiondefinition;

functiondefinition ::= type Ident "(" parameterlist? ")" "{" statementlist "}";
parameterlist    ::= type Ident ("," type Ident)*;
functioncall     ::= Ident "(" (assignment ("," assignment)*)? ")";

statementlist    ::= statement*;
block            ::= "{" statementlist "}";

statement        ::= ifstatement
                  | forstatement
                  | dowhilestatement ";"
                  | whilestatement
                  | returnstatement ";"
                  | print ";"
                  | declassignment ";"
                  | statassignment ";"
                  | functioncall ";"
                  | block
                  | ";"
                  ;

ifstatement      ::= "if" "(" assignment ")" statement ("else" statement)?;
forstatement     ::= "for" "("
                  (statassignment | declassignment) ";"
                  expr ";"
                  statassignment
                  ")" statement;
dowhilestatement ::= "do" statement "while" "(" assignment ")";
whilestatement   ::= "while" "(" assignment ")" statement;
returnstatement  ::= "return" assignment?;
print            ::= "print" "(" (assignment ("," assignment)*)? ")";
declassignment   ::= type Ident ("=" assignment)?;

type             ::= "bool" | "float" | "int" | "void";

statassignment   ::= Ident "=" assignment;
assignment       ::= Ident "=" assignment | expr;
expr             ::= simpexpr (
                  "==" simpexpr | "!=" simpexpr |
                  "<=" simpexpr | ">=" simpexpr |
                  "<" simpexpr | ">" simpexpr
                  )?;
simpexpr         ::= term ("+" term | "-" term | "||" term)*;
term             ::= factor ("*" factor | "/" factor | "&&" factor)*;
factor           ::= "-" factor
                  | IntLiteral
                  | FloatLiteral
                  | BoolLiteral
                  | StringLiteral
                  | Ident
                  | functioncall
                  | "(" assignment ")"
                  ;

```

Vorlage

Um Ihnen den Einstieg in diese Aufgabe zu vereinfachen, geben wir Ihnen ein Programmgerüst vor. Die Ordnerstruktur des entpackten Archives ist wie folgt:

```

.
├── src
│   ├── ast.rs
│   ├── lexer.rs
│   ├── lib.rs
│   ├── main.rs
│   └── parser.lalrpop
├── tests
│   ├── inputs
│   │   ├── ok
│   │   └── parser_err
│   ├── util
│   ├── lexer_tests.rs
│   ├── lexer_tests_extra.rs
│   └── parser.rs
├── Cargo.lock
├── Cargo.toml
└── build.rs

```

Die Dateien `main.rs`, `lib.rs`, `lexer.rs`, `ast.rs`, `build.rs` sowie die `Cargo.toml` müssen Sie nicht anpassen. Sie enthalten den Programmeinstiegspunkt (`main.rs`), exponieren die öffentliche Schnittstelle (`lib.rs`), den Lexer (`lexer.rs`) aus der vorigen Aufgabe, die Datenstrukturen des abstrakten Syntaxbaumes (`ast.rs`), das Buildsript (`build.rs`), welches zur Übersetzung des Parsers und der Generierung der Testfälle genutzt wird, und Metainformationen über das Projekt (`Cargo.toml`).

- Sie können anstelle der Vorlage des Lexers in `lexer.rs` Ihren eigenen Lexer aus der letzten Aufgabe einsetzen. Führen Sie in diesem Fall sicherheitshalber `cargo test` aus und schauen Sie auf die Ergebnisse von `lexer_tests.rs` und `lexer_tests_extra.rs` um sicher zu gehen, dass Ihr Lexer die richtigen Tokentypen erkennt.
- In `parser.lalrpop` befinden sich Tokendefinitionen sowie der Rumpf der Grammatik. Fügen Sie hier Ihre Grammatik-Regeln hinzu.
- Im `tests/inputs`-Ordner finden Sie Testfälle für syntaktisch korrekte (`ok`) und inkorrekte (`parser_err`) C1-Programme. Beim Aufruf von `cargo test` wird geprüft, ob Ihr Parser bei der syntaktischen Prüfung und der Konstruktion des abstrakten Syntaxbaumes zum gleichen Ergebnis wie unser Referenzparser kommt. Dafür müssen sowohl das Ergebnis (syntaktisch korrekt oder nicht), als auch die konkrete Instanz des abstrakten Syntaxbaumes übereinstimmen. Sollte das nicht der Fall sein, gibt der Testfall Ihnen die Unterschiede zurück.

Hinweise

- Machen Sie sich zunächst mit den Datenstrukturen in `ast.rs` vertraut, um ein Verständnis für den Zusammenhang dieser Strukturen zu entwickeln.
- Sie werden feststellen, dass die Strukturen des abstrakten Syntaxbaumes sich an den grammatikalischen Regeln der Sprache orientieren. Das ist natürlich kein Zufall und sollte Ihnen helfen den Metasymbolen die richtigen Datentypen zuzuordnen.
- Machen Sie sich auch mit der Syntax von `lalrpop` bekannt, etwa indem Sie sich das `lalrpop`-Handbuch (lalrpop.github.io/lalrpop) anschauen.
- Sie können in `lalrpop` die in der Grammatik eingesetzte EBNF-Notation verwenden.
- Leider können Sie keine Präzedenzen an Grammatikregeln annotieren, um `shift-reduce`-Konflikte auf diese Weise aufzulösen. Das wird Ihnen an einer berühmt-berüchtigten Stelle Probleme bereiten.
- Wenn Sie diese Stelle erreichen, werden Sie Ihre Grammatik umstrukturieren müssen. Überlegen Sie sich genau, welche Konfliktlösung (`shift` oder `reduce`) gewollt ist und bauen die Grammatik entsprechend um. Die mitgelieferten Testfälle helfen Ihnen dabei festzustellen, ob Sie die richtige Wahl getroffen haben.

Wir wünschen Ihnen viel Spaß und Erfolg bei der Bearbeitung dieses Übungsblattes.

Blatt 4 - Analyse (100 Punkte)

Fällig: Mo., 22. Juni 2026, 23:59

Lernziele

- Semantische Analyse mit geschichteten Sichtbarkeitsbereichen
- Umgang mit blockstrukturierter Symboltabelle
- Anreicherung eines abstrakten Syntaxbaumes mit semantischen Informationen

Generelles

In dieser Aufgabe implementieren Sie die semantische Analyse mithilfe einer blockstrukturierten Symboltabelle und reichern den abstrakten Syntaxbaum für unsere Sprache C1 mit semantischen Informationen an. Der validierte und annotierte Syntaxbaum wird auf dem nächsten (und letzten) Übungsblatt von Ihnen durch die Implementation eines Interpreters mit einer Ausführungssemantik ausgestattet.

Sie können sich aussuchen, ob Sie dieses Übungsblatt in Rust oder in C lösen möchten. Die Variante für C finden Sie hier (Moodle).

Allgemeine Hinweise zur Bearbeitung und zum Abgabemodus finden Sie auf dieser Seite (Moodle).

Aufgabenstellung

Wir geben Ihnen eine Vorlage mit dem Lexer, dem Parser, den Datenstrukturen des abstrakten Syntaxbaumes sowie einer blockstrukturierten Symboltabelle vor. Implementieren Sie die semantische Analyse in der gegebenen Vorlage `analysis.rs` unter Zuhilfenahme der zur Verfügung gestellten Symboltabelle. Die zu prüfende Programmsemantik finden Sie nachfolgend. Um Ihnen das Testen zu erleichtern, geben wir Ihnen eine Sammlung von fehlerhaften C1-Programmen mit, die viele (aber nicht alle) semantischen Fehlerquellen abdeckt.

Alle C1-Programme erfüllen über die syntaktischen Anforderungen hinaus auch noch die folgenden Regeln der statischen, also zur Übersetzungszeit geprüften, Semantik.

1. Sichtbarkeit

Ein Sichtbarkeitsbereich ist ein Abschnitt des Quellcodes, der zu folgenden Metasymbolen reduziert wird:

- `block`
- `forstatement`
- `functiondefinition`, wobei der Funktionsname Teil des äußeren Sichtbarkeitsbereiches ist

Die Grammatik erlaubt eine Schichtung von Sichtbarkeitsbereichen.

Es gelten folgende Regeln:

1. Es gibt eine parameterlose `main()`-Funktion mit dem Rückgabetyt `void`.
2. Funktionen sind im gesamten Programm sichtbar.
3. Variablen müssen vor ihrer Benutzung deklariert werden.
4. In jedem Sichtbarkeitsbereich darf ein Bezeichner höchstens ein Mal deklariert werden.
5. Bei der Namensauflösung von Bezeichnern werden die Sichtbarkeitsbereiche von innen nach außen durchsucht und der erste Treffer ausgewählt.

2. Typenkompatibilität

C1 unterstützt die implizite Konvertierung von `int` nach `float`. Ein Typ wird als kompatibel zu einem zweiten Typ bezeichnet, wenn er identisch oder in den anderen konvertierbar ist.

Es gelten folgende Regeln:

1. Die Bedingungen für while-, do while-, for- und if-Schleifen müssen boolesche Ausdrücke sein.
2. Die Ausdrücke in der print-Anweisung dürfen nicht vom Typ void sein.
3. Es dürfen keine Variablen und Parameter vom Typ void deklariert werden.
4. Funktionsaufrufe müssen parameterkonform sein, das bedeutet: Parameter- und Argumentlisten müssen die gleiche Länge haben; Argumenttypen müssen paarweise kompatibel mit den Parametertypen sein.
5. Der Typ eines Rückgabewertes muss kompatibel zum Rückgabotyp der umgebenden Funktion sein.
6. Funktionen mit dem Rückgabotyp void dürfen keine Ausdrücke zurückgeben, auch keine vom Typ void.
7. Zuweisungen dürfen nur an Variablenbezeichner erfolgen.
8. Der Typ der rechten Seite einer Zuweisung muss kompatibel zum Typ der linken Seite sein.

3. Ausdrücke und Operatoren

1. Der Funktionsaufrufoperator (()) darf nur auf Funktionsbezeichner angewandt werden.
2. Funktionsbezeichner sind nur nach der Anwendung des Funktionsaufrufoperators gültige Ausdrücke.
3. Die Operanden eines Vergleichsausdrucks müssen miteinander kompatibel und dürfen nicht vom Typ void sein.
4. Die Operanden in logischen Operationen müssen vom Typ bool sein.
5. Die Operanden in arithmetischen Operationen (Addition, Subtraktion, Multiplikation, Division und Negation) müssen vom Typ int oder float sein.
6. Der Ergebnistyp einer Zuweisung ist der Typ der Variablen auf der linken Seite.
7. Der Ergebnistyp einer Vergleichs- oder logischen Operation ist bool.
8. Der Ergebnistyp einer arithmetischen Operation ist float, wenn einer der beteiligten Operanden vom Typ float ist, und ansonsten int.

Vorlage

Um Ihnen den Einstieg in diese Aufgabe zu vereinfachen, geben wir Ihnen ein Programmgerüst vor. Die Ordnerstruktur des entpackten Archives ist wie folgt:

```
.
├── src
│   ├── analysis
│   │   └── symtab.rs
│   ├── analysis.rs
│   ├── ast.rs
│   ├── lexer.rs
│   ├── lib.rs
│   ├── main.rs
│   └── parser.lalrpop
├── tests
├── Cargo.lock
├── Cargo.toml
└── build.rs
```

Dateien, die Sie nicht anpassen müssen:

- `ast.rs`: Datenstrukturen des abstrakten Syntaxbaumes.
- `lexer.rs`: Der Logos-Lexer aus den vorigen Aufgaben.
- `lib.rs`: Exponiert die öffentliche Schnittstelle.
- `symtab.rs`: Blockstrukturierte Symboltabelle für die Nutzung im Rahmen dieser Aufgabe.
- `main.rs`: Enthält den Programmeinstiegspunkt.

- `parser.lalrpop`: Parser aus der vorigen Aufgabe, mit Syntaxanalyse und Konstruktion des abstrakten Syntaxbaumes.
- `Cargo.toml`: Enthält Meta-Informationen über das Projekt.
- `build.rs`: Das Buildsript zur Übersetzung von Lexer und Parser sowie zur Ausführung der Testfälle.

Ordner, die Sie in der Entwicklungsphase ignorieren können:

- `tests`: Enthält Testfälle und – in `tests/inputs` – Testeingaben.

Diese Struktur soll Ihnen dabei helfen, sich schnell zurechtzufinden und sich auf die relevanten Teile des Projekts zu konzentrieren.

Aufgabe

1. Implementieren Sie Tests für die semantischen Regeln in `analysis.rs` und sammeln Sie semantische Informationen über Bezeichner mit der mitgelieferten Symboltabelle.
2. Die Datei `parser.lalrpop` enthält bereits die syntaktische Analyse und den abstrakten Syntaxbaum (AST) aus der letzten Aufgabe. Sie können Ihre eigene Syntaxanalyse einfügen und sollten dann `cargo test` ausführen, um die Grammatik zu überprüfen.
3. Sie können Ihren eigenen Lexer anstelle der Vorlage in `lexer.rs` verwenden und sollten auch hier `cargo test` ausführen, um die Tokentypen zu überprüfen.
4. Im Ordner `tests/inputs` finden Sie Testfälle für: syntaktisch und semantisch korrekte Programme (`ok` und `interpreter_err`); syntaktisch inkorrekte Programme (`parser_err`); semantisch inkorrekte Programme (`analysis_err`).

`cargo test` prüft, ob Ihr Parser syntaktisch und semantisch korrekt ist und ob der AST mit dem Referenzparser übereinstimmt. Bei Abweichungen werden die Differenzen in die Konsole ausgegeben, die Sie zum Debuggen nutzen können.

Hinweise

- Annotieren Sie die Tests der semantischen Bedingungen in die `Analyzer::visit_*()`-Methoden in der Datei `analysis.rs`.
- Bestimmen Sie für jede AST-Struktur die notwendigen semantischen Informationen und nutzen Sie die Symboltabelle sowie uninitialisierte Teile des AST zur Berechnung.
- Gehen Sie systematisch alle Produktionen durch, berechnen Sie die semantischen Informationen (z.B. Datentypen, Sichtbarkeitsbereiche, Bezeichnerdeklarationen) und speichern Sie diese im AST.
- Ergänzen Sie dann die semantischen Tests entsprechend den Regeln. Jede Regel sollte zu mindestens einem Test oder einer Wertberechnung führen.
- Implementieren Sie erforderliche Hilfsfunktionen.

Wir wünschen Ihnen viel Spaß und Erfolg bei der Bearbeitung dieses Übungsblattes.

Blatt 5 - Interpreter

Lernziele

- Interpretation einer annotierten Zwischenrepräsentation
- Umsetzung von Laufzeitsemantik
- Prüfung auf undefiniertes Verhalten

Generelles

In dieser Aufgabe implementieren Sie den Interpreter für den annotierten und bezüglich der statischen Semantik validierten abstrakten Syntaxbaum, führen diesen aus und überprüfen dabei die dynamische Semantik auf undefiniertes Verhalten. Diese Aufgabe müssen Sie lösen, wenn Sie von der SPO 2015 in die SPO 2022 wechseln wollen.

Sie können sich aussuchen, ob Sie dieses Übungsblatt in Rust oder in C lösen möchten. Die Variante für C finden Sie hier (Moodle).

Allgemeine Hinweise zur Bearbeitung und zum Abgabemodus finden Sie auf dieser Seite (Moodle).

Aufgabenstellung

Wir geben Ihnen eine Vorlage mit dem Lexer, dem Parser, den Datenstrukturen des abstrakten Syntaxbaumes, dem Ergebnis der semantischen Analyse sowie einiger Hilfsfunktionen vor. Implementieren Sie die Interpretation in der Datei `interpreter.rs` unter Nutzung der vorgegebenen Hilfsfunktionen und -strukturen. Prüfen Sie während der Ausführung den Programmzustand auf undefiniertes Verhalten und brechen Sie ggf. die Ausführung mit einer Fehlermeldung ab. Um Ihnen das Testen zu erleichtern, geben wir Ihnen eine Sammlung von fehlerhaften C1-Programmen mit, die viele (aber nicht alle) Fehlerquellen der dynamischen Semantik abdeckt.

Definiertes Verhalten

Die dynamische oder Laufzeitsemantik von C1-Programmen orientiert sich an der Laufzeitsemantik von C-Programmen:

1. Alle globalen Variablen mit Initialisierungsausdruck werden in der Reihenfolge ihrer Deklaration vor dem Start der Programmausführung initialisiert.
2. Die Programmausführung startet am Anfang von `main()`.
3. Die Kontrollstrukturen `if`, `while`, `do while` und `for` haben dieselbe Bedeutung wie in C.
4. Die Liste von Anweisungen in Codeblöcken werden der Reihe nach ausgeführt.
5. Die `print`-Anweisung endet stets mit einem Zeilenende und produziert folgende Ausgaben: Zeichenkettenlitterale werden unverändert ausgegeben. Für boolesche Ausdrücke mit dem Wert 1 ist die Ausgabe `"true"`, mit dem Wert 0 ist sie `"false"`. Für integrale Ausdrücke wird der Wert in Dezimaldarstellung ausgegeben. Für gewöhnliche Fließkommaausdrücke wird der Wert in Dezimal- oder wissenschaftlicher Notation ausgegeben, je nachdem welche Darstellung kürzer ist. Für ungewöhnliche Fließkommawerte (NaN und $\pm$ unendlich) wird `"nan"` ausgegeben, falls der Wert ungleich sich selbst ist, sowie `"inf"` für den größten und `"-inf"` für den kleinsten Wert.
6. Funktionsrufe, die als Anweisungen auftauchen, also deren Rückgabewert keiner Variablen zugewiesen wird, werden ausgeführt und deren Ergebnis anschließend verworfen.
7. Arithmetische, logische und Vergleichsoperationen haben dieselbe Bedeutung wie in C, werden jedoch in Abweichung zu C stets von links nach rechts berechnet.
8. Die Werte der Ausdrücke in Funktionsargumenten werden von links nach rechts berechnet.
9. Variablenzuweisungen erlangen direkt im Anschluss der Berechnung des Ausdrucks auf der rechten Seite Gültigkeit.
10. Beim Lesen von Variablen wird der zuletzt geschriebene Wert zurückgegeben.

Undefiniertes Verhalten

Folgendes Programmverhalten ist undefiniert:

1. Der Überlauf von `int`-Werten durch Addition, Subtraktion, Multiplikation, Division oder Negation ist undefiniert.
2. Das Ergebnis einer Integer-Division durch 0 ist undefiniert.
3. Das Lesen aus uninitialisierten Variablen ist undefiniert.
4. Die Rückkehr aus Funktionen mit Rückgabetypp nicht `void` ohne Nutzung der `return`-Anweisung ist undefiniert, auch wenn der Wert danach verworfen würde.
5. Das Abarbeitungsergebnis rekursiver Funktionen, die während der Ausführung sämtlichen verfügbaren Arbeitsspeicher erschöpfen, ist undefiniert.

Die fünfte Regel soll nicht dynamisch getestet werden. Stattdessen ist es zulässig, wenn dieser Fall bei der Ausführung zu undefiniertem Verhalten führt.

Vorlage

Um Ihnen den Einstieg in diese Aufgabe zu vereinfachen, geben wir Ihnen ein Programmgerüst vor. Die Ordnerstruktur des entpackten Archives ist wie folgt:

```
.
├── src
│   ├── analysis
│   │   └── symtab.rs
│   ├── interpreter
│   │   └── vm.rs
│   ├── analysis.rs
│   ├── ast.rs
│   ├── interpreter.rs
│   ├── lexer.rs
│   ├── lib.rs
│   ├── main.rs
│   └── parser.lalrpop
├── tests
├── Cargo.lock
├── Cargo.toml
└── build.rs
```

Von diesen Dateien müssen Sie prinzipiell nur die `interpreter.rs` bearbeiten, welche wir für Sie mit `todo!()` versehen haben. Die anderen Dateien enthalten:

- `analysis.rs`: Prüfung der statischen Semantik und die Anreicherung des abstrakten Syntaxbaumes mit den berechneten semantischen Informationen.
- `vm.rs`: Enthält eine virtuelle Maschine mit Methoden zum Schreiben und Lesen von Variablen und der Verwaltung von Stack-Frames. Definiert Datentypen für stark-typisierte Speicherstellen mit Schnittstellen zum abstrakten Syntaxbaum.
- `ast.rs`: Datenstrukturen des abstrakten Syntaxbaumes.
- `lexer.rs`: Der Logos-Lexer aus den vorigen Aufgaben.
- `lib.rs`: Exponiert die öffentliche Schnittstelle.
- `parser.lalrpop`: Prüfung der Korrektheit von Syntax sowie die Konstruktion des abstrakten Syntaxbaumes.
- `symtab.rs`: Blockstrukturierte Symboltabelle für die Nutzung im Rahmen der semantischen Analyse.
- `main.rs`: Enthält den Programmeinstiegspunkt.
- `Cargo.toml`: Enthält Meta-Informationen über das Projekt.
- `build.rs`: Das Buildskript zur Übersetzung des Parsers sowie zur Ausführung der Testfälle.

Im `tests`-Ordner finden Sie Testeingaben, welche Sie mit `cargo test` ausführen können. Eventuelle Differenzen zur Referenzlösung werden beim Testen für Sie zu Debugzwecken ausgegeben.

Hinweise

- Kümmern Sie sich zunächst kleinschrittig um die Umsetzung der definierten Teile der dynamischen Semantik, bevor Sie die Fehlerbehandlung für undefiniertes Verhalten einbauen.
- Dazu ist es empfehlenswert mit sehr einfachen C1-Programmen - etwa `tests/input/ok/hello.c1` - anzufangen und nur diesen einen Codepfad (Einstieg in `main()`, `print`-Anweisung für Zeichenkettenlitterale) vollständig zu implementieren, bevor Sie sich an komplexere AST-Knoten wagen. Diesen Schritt haben wir Ihnen in der Vorlage bereits abgenommen.
- Um Integer-Überläufe zu erkennen, nutzen Sie die `checked_*()`-Methoden für Integer-Zahlen aus der Rust-Standardbibliothek.
- Im Fließkommaformat gibt es keinen Überlauf und auch Division durch 0 ist definiert.
- Die virtuelle Maschine unterstützt Methoden, die Sie für den Aufbau des Callstacks nutzen können.
- Bei der Ausgabe der Fließkommalitterale kann `println!()` mit dem Debug-Formatspezifikator `"{:?}"` verwendet werden.

Wir wünschen Ihnen viel Erfolg beim Lösen dieser Aufgabe und hoffen, dass Sie genau so viel Spaß bei der Umsetzung dieses Projektes hatten, wie wir bei der Konzeption!